



Міністерство освіти і науки України
Національний технічний університет України
“Київський політехнічний інститут імені Ігоря Сікорського”
Факультет інформатики та обчислювальної техніки
Кафедра інформаційних систем та технологій

Лабораторна робота №2
Проектування та дослідження програмних систем
зі штучним інтелектом

*Тема: «Реалізація та дослідження базових архітектур
нейронних мереж для моделювання функцій двох змінних»*

Виконала
студентка групи ІК-51мп:
Яковенко Д. О.

ЗМІСТ

Практичне завдання.....	2
Хід роботи.....	3
Обрана функція двох змінних	3
Архітектури досліджуваних нейронних мереж.....	4
Код і отримані результати	6
Відповіді на питання	10
Висновки.....	11

Практичне завдання

Мета роботи: Дослідити архітектуру та принцип роботи нейронних мереж. За допомогою нейронних мереж змодельовати функцію двох змінних.

Завдання: Написати програму, що реалізує нейронні мережі для моделювання функції двох змінних. Функцію двох змінних, типу $f(x,y) = x^2 + y^2$, обрати самостійно. Промодельовати на невеликому відрізку, скажімо від 0 до 10.

Дослідити вплив кількості внутрішніх шарів та кількості нейронів на середню відносну помилку моделювання для різних типів мереж (feed forward backprop, cascade - forward backprop, elman backprop):

1. Тип мережі: feed forward backprop:

- а) 1 внутрішній шар з 10 нейронами;
- б) 1 внутрішній шар з 20 нейронами;

2. Тип мережі: cascade - forward backprop:

- а) 1 внутрішній шар з 20 нейронами;
- б) 2 внутрішніх шари по 10 нейронів у кожному;

3. Тип мережі: elman backprop:

- а) 1 внутрішній шар з 15 нейронами;
- б) 3 внутрішніх шари по 5 нейронів у кожному;

4. Зробити висновки на основі отриманих даних.

Хід роботи

Обрана функція двох змінних

Для виконання лабораторної роботи було самостійно обрано нелінійну функцію двох змінних, яка має вигляд чаші (параболоїда обертання) з мінімумом у точці (5,5):

$$f(x, y) = (x - 5)^2 + (y - 5)^2$$

Моделювання проводилося на відрізку від 0 до 10 для обох змінних (x та y). Для формування навчальної вибірки було створено сітку значень (20x20 точок).

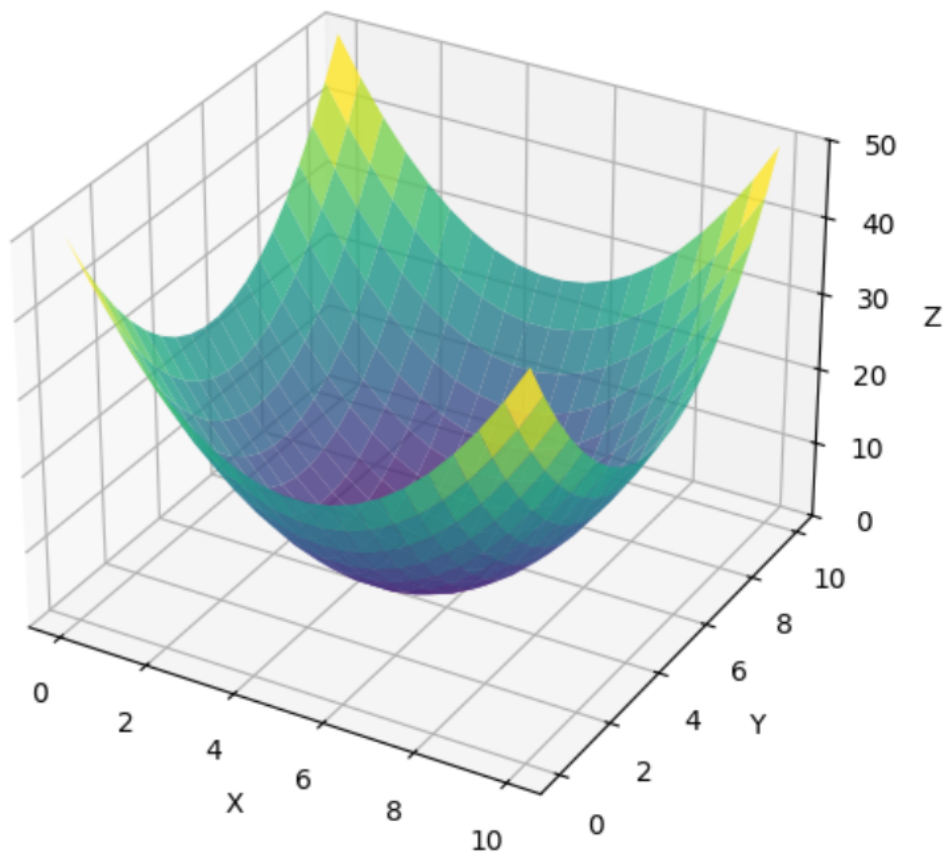


Рис. 1. – Графік оригінальної функції

Архітектури досліджуваних нейронних мереж

У роботі було реалізовано та досліджено три типи архітектур з різними конфігураціями прихованих шарів:

1. **Feed Forward (Багатошаровий перцептрон):** Класична мережа прямого поширення сигналу. Дані послідовно проходять від входу через приховані шари до виходу.

- Конфігурація *a*: 1 прихований шар з 10 нейронами.
- Конфігурація *b*: 1 прихований шар з 20 нейронами.

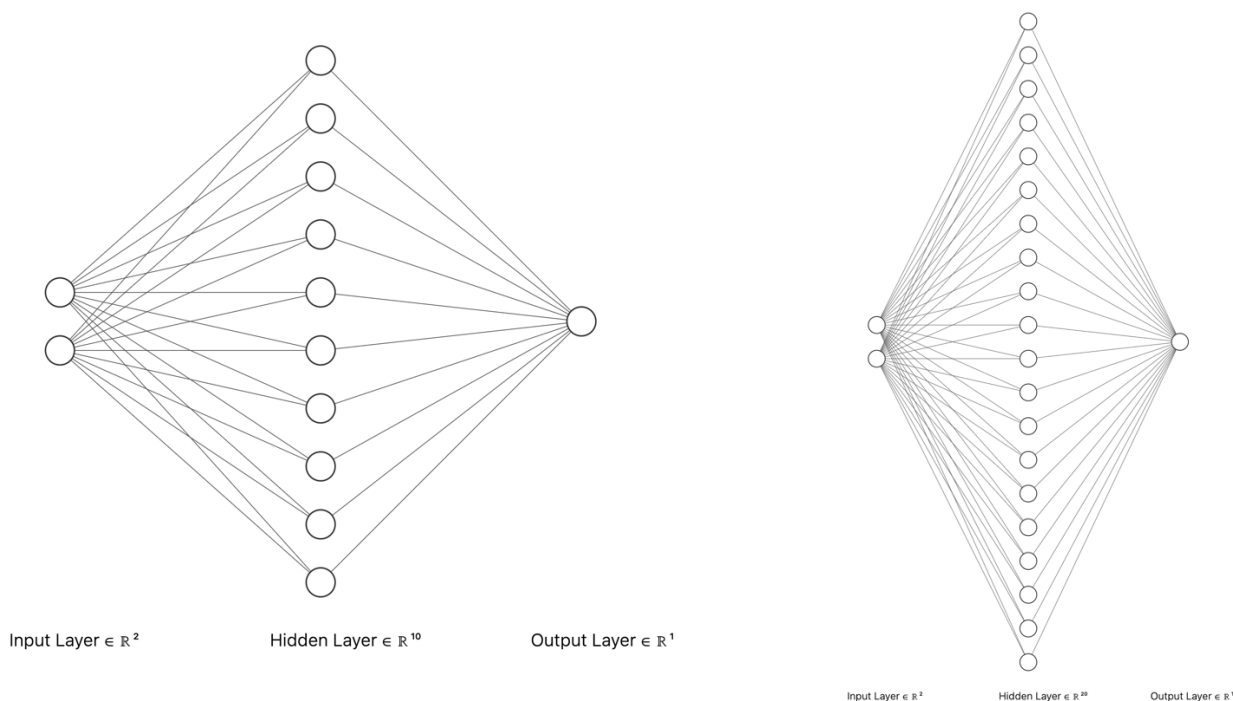


Рис. 2. – Багатошаровий перцептрон
(ліворуч – Конфігурація *a*, праворуч – Конфігурація *b*)

2. **Cascade-Forward:** Модифікація мережі прямого поширення, у якій кожен шар отримує вхідні сигнали не лише від попереднього шару, але й безпосередньо від початкових входів.

- Конфігурація *a*: 1 внутрішній шар з 20 нейронами.
- Конфігурація *b*: 2 внутрішніх шари по 10 нейронів у кожному.

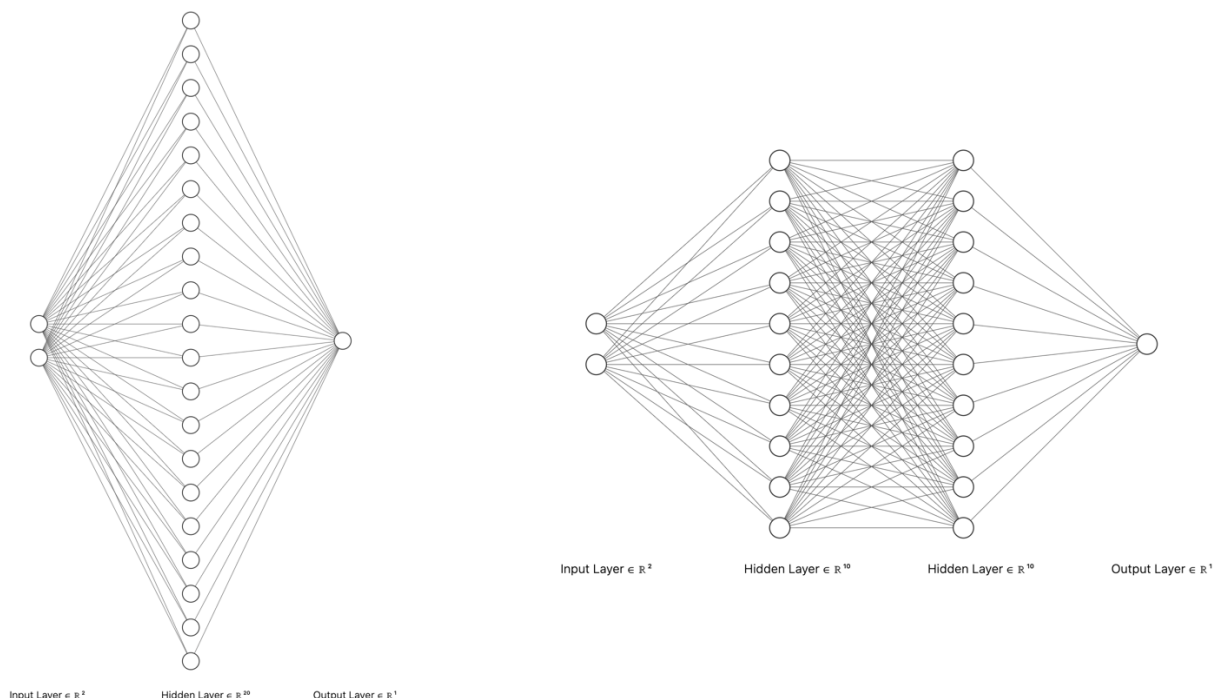


Рис. 3. – Cascade-Forward

3. **Elman Net (Рекурентна мережа):** Мережа з частковим зворотним зв'язком, яка містить так званий "контекстний шар" для збереження стану з попереднього такту.

- Конфігурація *a*: 1 внутрішній шар з 15 нейронами.
- Конфігурація *b*: 3 внутрішніх шари по 5 нейронів у кожному.

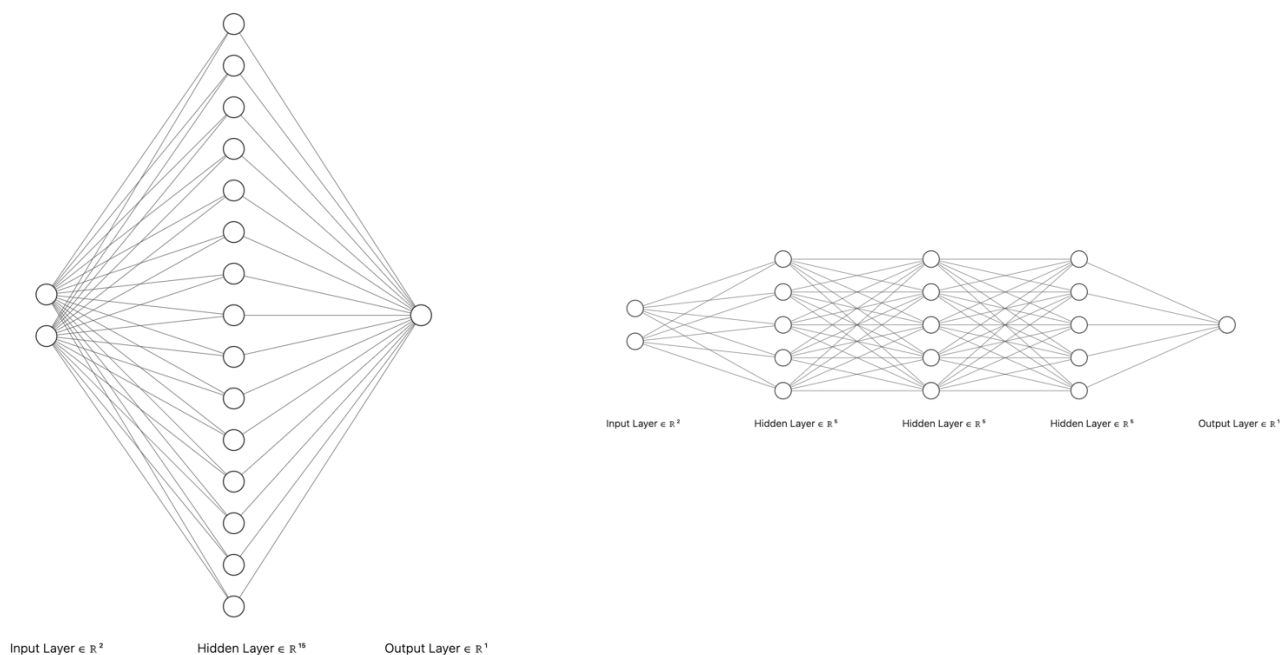


Рис. 4. – Рекурентна мережа

Усі мережі використовують гіперболічний тангенс ($\tanh$) як функцію активації прихованих шарів.

Код і отримані результати

Код:

```
# =====
# 1. Генерація даних
# =====
x_val = np.linspace(0, 10, 20)
y_val = np.linspace(0, 10, 20)
X, Y = np.meshgrid(x_val, y_val)

# Функція:  $f(x, y) = (x - 5)^2 + (y - 5)^2$ 
Z = (X - 5)**2 + (Y - 5)**2

inputs = np.vstack([X.ravel(), Y.ravel()]).T
targets = Z.ravel()[ :, None]

# Нормалізація цільової змінної
z_max = targets.max()
targets_norm = targets / z_max

# =====
# 2. Архітектури мереж (Keras)
# =====

# 1. Feed Forward (Послідовна модель)
def build_ff(hidden_size):
    return Sequential([
        Input(shape=(2,)),
        Dense(hidden_size, activation='tanh'),
        Dense(1)
    ])

# 2. Cascade-Forward
def build_cf(layers):
    inp = Input(shape=(2,))
    h1 = Dense(layers[0], activation='tanh')(inp)

    if len(layers) == 1:
        concat = Concatenate()([inp, h1])
        out = Dense(1)(concat)
    else:
        concat1 = Concatenate()([inp, h1])
        h2 = Dense(layers[1], activation='tanh')(concat1)
        concat2 = Concatenate()([inp, h1, h2])
        out = Dense(1)(concat2)

    return Model(inputs=inp, outputs=out)
```

```

# 3. Elman Net
def build_elman(layers):
    model = Sequential()
    model.add(Input(shape=(1, 2)))

    if len(layers) == 1:
        model.add(SimpleRNN(layers[0], activation='tanh'))
    else:
        model.add(SimpleRNN(layers[0], activation='tanh', return_sequences=True))
        model.add(SimpleRNN(layers[1], activation='tanh', return_sequences=True))
        model.add(SimpleRNN(layers[2], activation='tanh'))

    model.add(Dense(1))
    return model

# =====
# 3. Налаштування експериментів
# =====
configs = {
    "Feed Forward (10 neurons)": build_ff(10),
    "Feed Forward (20 neurons)": build_ff(20),
    "Cascade-Forward (20 neurons)": build_cf([20]),
    "Cascade-Forward (10+10 neurons)": build_cf([10, 10]),
    "Elman Net (15 neurons)": build_elman([15]),
    "Elman Net (3x5 neurons)": build_elman([5, 5, 5])
}

epochs = 100
results = {}

def custom_accuracy(y_true, y_pred):
    mae = tf.reduce_mean(tf.abs(y_pred - y_true))
    return tf.maximum(0.0, 1.0 - mae) * 100.0

# =====
# 4. Навчання та збір метрик
# =====
print("Розпочинаємо навчання мереж...")
for name, model in configs.items():
    model.compile(optimizer=tf.keras.optimizers.Adam(learning_rate=0.02),
                  loss='mse',
                  metrics=[custom_accuracy])

    # Рекурентна мережа потребує 3D формату даних
    x_train = inputs.reshape(-1, 1, 2) if "Elman" in name else inputs

    history = model.fit(x_train, targets_norm, epochs=epochs, verbose=0)

    final_preds = model.predict(x_train, verbose=0) * z_max
    acc_history = history.history['custom_accuracy']

    results[name] = {
        "losses": history.history['loss'],
        "accuracies": acc_history,
        "predictions": final_preds.reshape(X.shape),
        "final_acc": acc_history[-1]
    }
print(f"[{name}] Навчання завершено. Фінальна точність: {acc_history[-1]:.2f}%")

```

Результати:

Розпочинаємо навчання мереж...

[Feed Forward (10 neurons)] Навчання завершено. Фінальна точність: 95.92%

[Feed Forward (20 neurons)] Навчання завершено. Фінальна точність: 95.78%

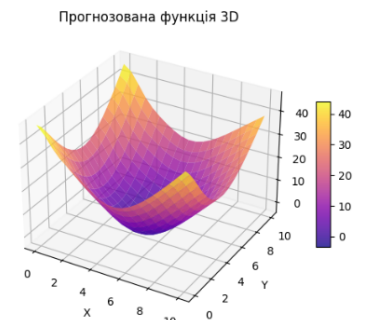
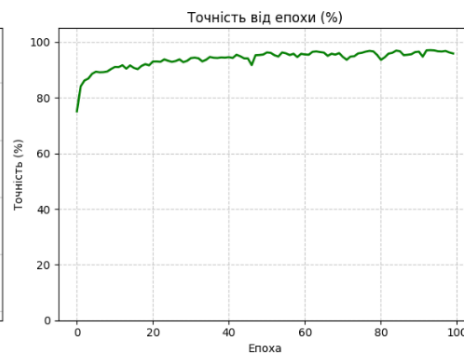
[Cascade-Forward (20 neurons)] Навчання завершено. Фінальна точність: 98.07%

[Cascade-Forward (10+10 neurons)] Навчання завершено. Фінальна точність: 98.95%

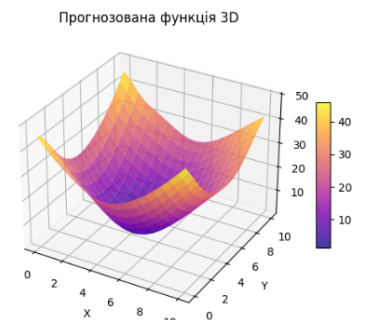
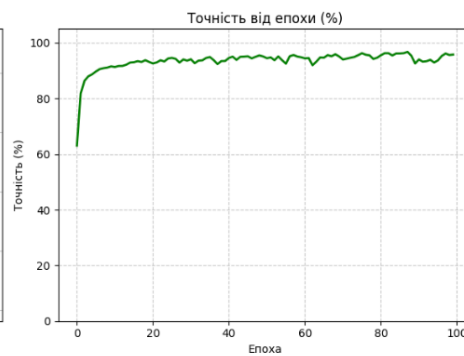
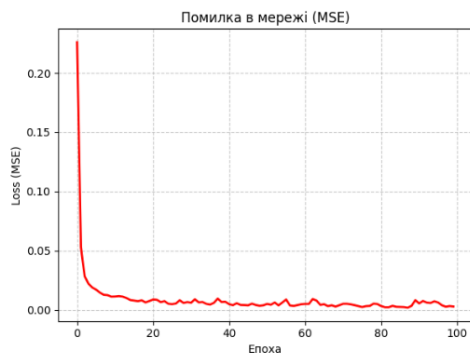
[Elman Net (15 neurons)] Навчання завершено. Фінальна точність: 96.41%

[Elman Net (3x5 neurons)] Навчання завершено. Фінальна точність: 94.93%

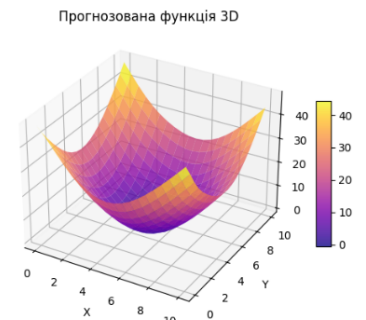
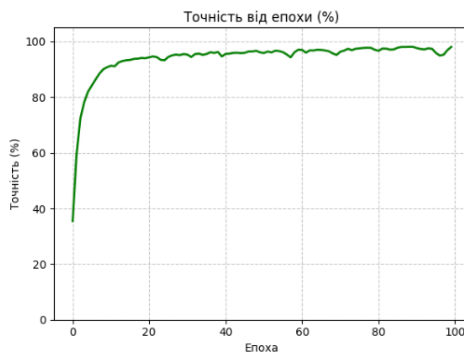
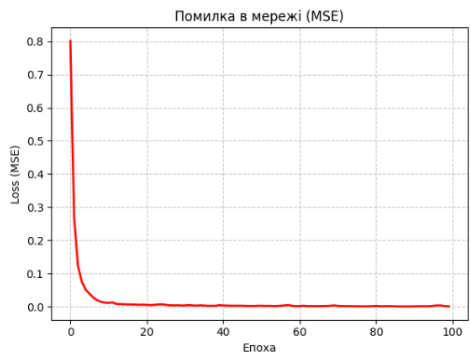
Аналіз мережі: Feed Forward (10 neurons)



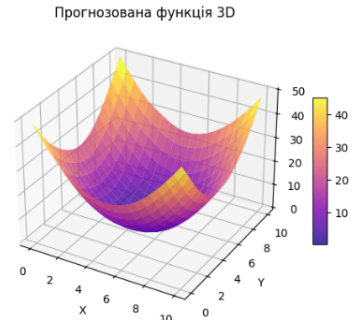
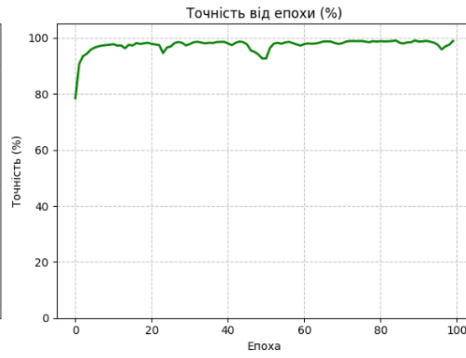
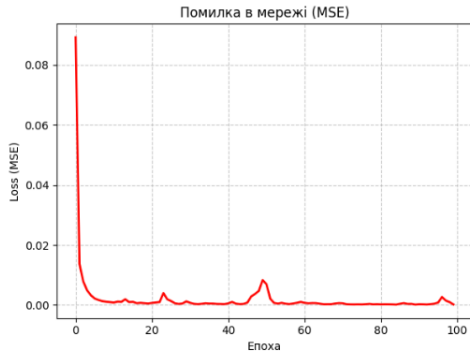
Аналіз мережі: Feed Forward (20 neurons)



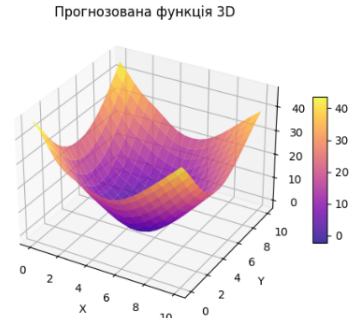
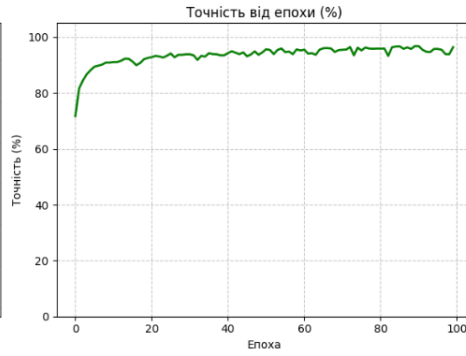
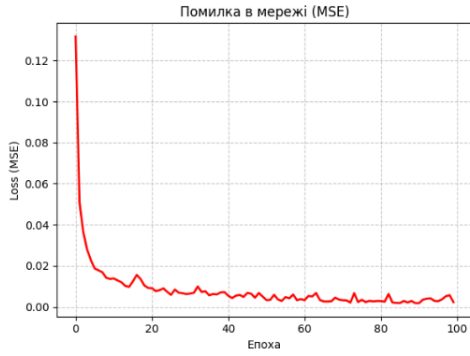
Аналіз мережі: Cascade-Forward (20 neurons)



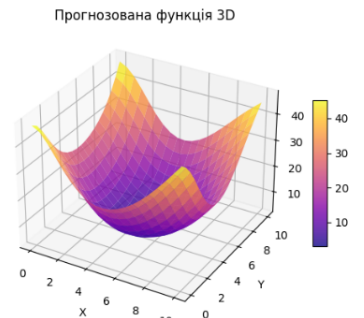
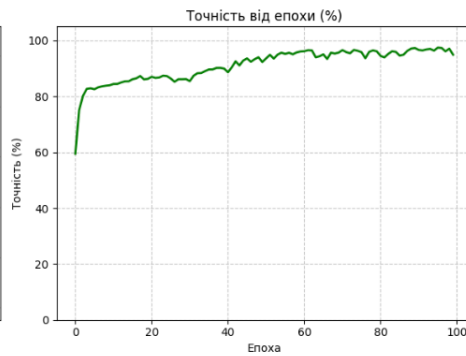
Аналіз мережі: Cascade-Forward (10+10 neurons)



Аналіз мережі: Elman Net (15 neurons)



Аналіз мережі: Elman Net (3x5 neurons)

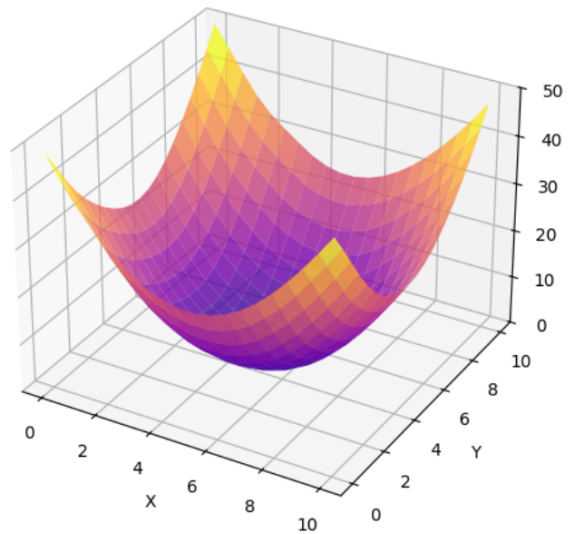
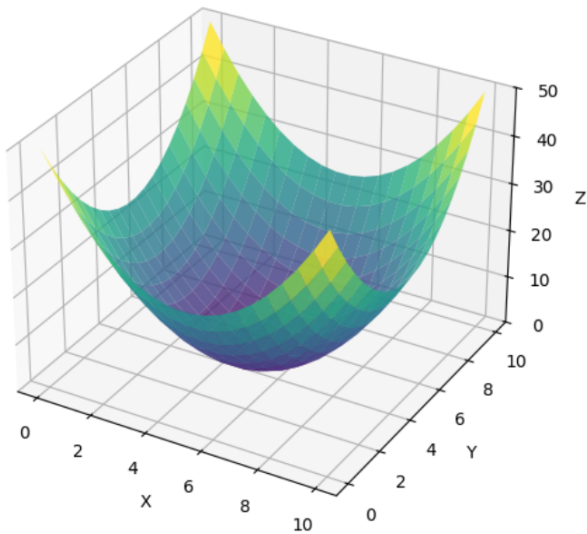


ПОРІВНЯННЯ: Оригінал vs Найкраща модель (Cascade-Forward (10+10 neurons))

Загальна похибка моделювання: 0.85%

Оригінальна функція

Прогнозована функція



Відповіді на питання

1. **ШНМ** – це математична модель (а також її програмна чи апаратна реалізація), побудована за принципом організації та функціонування біологічних нейронних мереж живих організмів. Вона складається з множини з'єднаних між собою простих обчислювальних вузлів (штучних нейронів).
2. **Штучний нейрон складається з** входу, активаційної функції, вагових коефіцієнтів, зміщення, суматора.
3. **Архітектура нейронної мережі** – це принцип за яким буде здійснено побудову цієї мережі, її топологія, тобто кількість шарів і нейронів в кожному шарі і структура зв'язків між ними.
4. Є мережі **рекурентні, прямого поширення, каскадні мережі**. Всі відрізняються способом передачі інформації. ^[1]Прямого поширення – сигнал іде строго від входу до виходу, без зворотних зв'язків; ^[2]каскадні – подібні до мереж прямого поширення, але кожен шар отримує вхідні сигнали не лише від попереднього шару, а й *безпосередньо від початкових входів* та всіх попередніх шарів; ^[3]рекурентні – мають зворотні зв'язки (пам'ять), що дозволяє враховувати попередні стани при обробці послідовностей.
5. **Активаційна функція** визначає вихідне значення нейрона на основі його зваженої суми. Її **головна мета – внесення нелінійності** в модель. Без нелінійної активаційної функції багатошарова мережа працювала б просто як одна велика лінійна регресія і не змогла б вирішувати складні задачі.
6. Гіперболічний тангенс, сігмоїда, ReLU, Softmax.
7. **Навчання** – це ітеративний процес, метою якого є підібрати такі ваги, що будуть коректно вирішувати поставлену задачу.
8. **З учителем** – використовуються заздалегідь відомі правильні значення і порівнюється передбачення мережі з цим значенням, знаходиться похибка і поширюється, щоб мережа розуміла помилку. **Без учителя** – немає відомих значень, мережа вручну шукає патерни, які б задовольняли рішення. **З підкріпленням** – мережа не має готових відповідей, але отримує "винагороди" або "штрафи" за свої дії, намагаючись максимізувати сумарну винагороду.
9. **Помилка (функція втрат)** – це математична оцінка того, наскільки сильно передбачення мережі відрізняється від правильного значення

10. **Корекція ваг здійснюється** за допомогою методів градієнтного спуску. Мережа обчислює часткову похідну (градієнт) функції втрат по кожній вазі. Ваги оновлюються в напрямку, протилежному градієнту, за формулою:

$$w_{new} = w_{old} - \eta \cdot \nabla L,$$

де η – швидкість навчання (*learning rate*)

11. **Алгоритм зворотного поширення помилки** – це ефективний метод обчислення градієнтів функції втрат для всіх ваг мережі. Він використовує математичне правило ланцюга (chain rule), поширюючи інформацію про помилку від вихідного шару назад до вхідного, дозволяючи точно визначити, наскільки кожна окрема вага вплинула на загальну помилку.

12. Здатність до узагальнення, відмовостійкість, паралельність обчислень.

13. Обробка природної мови, аналіз і прогнозування, медична діагностика, автономне керування транспортом

14. **Головна перевага** – відсутність необхідності вручну прописувати правила вирішення задачі (наприклад, вручну писати фільтри для пошуку контурів).

Нейромережа сама автоматично витягує ознаки з даних і знаходить приховані закономірності, які людині було б складно формалізувати.

15. Black box (важко інтерпретувати, чому мережа прийняла саме таке рішення), залежність від даних, схильність до перенавчання, високі вимоги до обчислювальних ресурсів

Висновки

У ході виконання лабораторної роботи було створено та навчено нейромережеві моделі для апроксимації функції двох змінних. За допомогою бібліотеки Keras було досліджено три типи архітектур (Feed Forward, Cascade-Forward, Elman Net) та проаналізовано їхню ефективність після 100 епох навчання.

На основі отриманих результатів можна зробити наступні висновки:

- Feed Forward (точність ~95.8% – 95.9%). Класична мережа прямого поширення продемонструвала досить високу точність. При цьому збільшення кількості нейронів в одному шарі з 10 до 20 не дало приросту точності, що свідчить про межу можливостей для простої архітектури на фіксованій кількості епох.

- Cascade-Forward (точність 98.07% – 98.95%). Цей тип мережі виявився найефективнішим. Завдяки прямим зв'язкам, що передають вхідний сигнал до виходу оминаючи приховані шари, мережа чудово вловила нелінійність функції. Архітектура з двома прихованими шарами (10+10 нейронів) показала найвищий результат.
- Elman Net (точність 94.93% – 96.41%). Застосування рекурентної мережі до статичної задачі виявилось недоцільним. Наявність контекстного шару (пам'яті) не допомогла, а ускладнення архітектури до трьох шарів по 5 нейронів дало найгірший показник точності в експерименті (94.93%) через зайве збереження "стану".

Тому найкращою моделлю для моделювання обраної функції стала мережа Cascade-Forward (10+10 нейронів). Її прогнозована поверхня практично ідентична оригіналу, а загальна похибка моделювання становить лише 0.85%, що є відмінним результатом для даної задачі.